

CONFIDENTIAL · INTERNAL

Quest HQ System Audit Report

A comprehensive review of the Command Center platform — security, correctness, performance, architecture, and user experience — written for both business and engineering readers.

PREPARED

28 Jul 2026

METHOD

OOAD audit,
6 review tracks

SCOPE

Full stack
(web · API · DB)

STATUS

1 issue already
fixed

1 • Executive summary

In plain terms: the foundations of Quest HQ are strong, but a handful of issues can quietly corrupt data or confuse users, and the app is doing far more work behind the scenes than it needs to.

We reviewed the entire Quest HQ system across six independent tracks — client (browser) security, back-end & database security, software architecture, correctness/bugs, performance, and user experience/accessibility. Every issue in this report was confirmed by reading the actual code, not guessed.

The good news. The parts that protect customer data are well built. The back-end and database enforce strict tenant isolation, block cross-company access, verify payment webhooks, and harden their database functions. Our security review found **no critical break-ins** — no way for one company to read another's data, no injection holes, no leaked secrets in the code.

Where to focus. The risk lives in three areas: **(1)** the in-progress "contact filter rail" ships buttons that look clickable but do nothing; **(2)** several paths can silently corrupt or lose data (a spreadsheet import that puts phone numbers in the email field; a text message that can vanish without warning); and **(3)** the app rebuilds its *entire* screen on every tiny change, which wastes performance and interrupts users mid-typing.

What we counted

TOTAL FINDINGS

30

across 6 review tracks

URGENT (P0 / P1)

12

5 ship-blockers · 7 high

ALREADY FIXED

1

broken stylesheet rule

2 · How to read this report

Each finding is written twice: a one-line **“What it means”** for everyone, and the technical detail for engineers. Issues are grouped by **priority**, not by team, so the most important work is always at the top.

Priority levels

LEVEL	MEANING	ACT BY
P0	Ship-blocker	Before the current work is released — active breakage, data corruption, or a security hole.
P1	High	Next work cycle — real bugs or risks users/customers will hit.
P2	Medium	Scheduled hardening — fold into related feature work.
P3	Architecture	Long-term health — reduces future bugs and speeds up the team. Incremental.

The six tracks we reviewed

Client security

Can a malicious input run code in a user's browser?

Back-end & database security

Can someone reach data or actions they shouldn't?

Correctness / bugs

Does the software do the right thing with real-world data?

Performance & redundancy

Is the app fast, or doing wasted work?

Architecture (OOAD)

Is the code organized so the team can change it safely?

UX & accessibility

Is it usable, clear, and reachable by keyboard/screen readers?

3 · System health scorecard

A quick read on each area. "Grade" reflects current risk, not effort spent.

AREA	GRADE	SUMMARY
Back-end & DB security	Strong	Strict tenant isolation, verified webhooks, hardened DB functions. No critical holes.
Client (browser) security	Good*	Consistent output escaping; one stored-XSS gap via imported app files.
Correctness	At risk	Data-corruption on import, a daily date off-by-one, and two silent data-loss paths.
Performance	At risk	Whole-screen rebuild on every change; the same data re-computed dozens of times.
UX & accessibility	At risk	Dead controls in new work; keyboard users can't select or edit rows.
Architecture	Aging	One 38k-line file holds almost everything; safe to improve incrementally.

The system at a glance

MAIN CODE FILE 38,781 lines in one file (<code>main.js</code>)	SCREEN REBUILDS TRIGGERED FROM ~549 places in the code	SHARED STATE FIELDS 228 in one global object
---	---	---

One issue was already resolved during this audit: a new stylesheet block had been pasted inside another unclosed rule, which would have left the new contact filter rail completely unstyled and broken a task badge. Fixed and verified with a clean production build.

4 · Priority 0 — Ship-blockers 5

These should be resolved before the current contact-rail work is released. They are active breakage, data corruption, or a security hole.

P0-1 “Team / Private” tabs do nothing

UX · IN-PROGRESS WORK

WHAT IT MEANS

The new contact panel shows a “Team / Private” switch that looks fully functional but is not connected to anything. Clicking it gives no response, which reads as a broken app.

TECHNICAL DETAIL

The buttons are rendered with active/inactive styling but carry no `data-action` and have no handler anywhere. `src/main.js:8400`

FIX

Wire to a real state key with a handler and filtered data, or remove the tabs until the feature exists.

P0-2 “Show more” is dead — some filters are unreachable

UX · IN-PROGRESS WORK

WHAT IT MEANS

Each filter group only shows the first 7 values; the rest hide behind a “Show more” link that doesn't work. Because this panel is now the *only* way to filter, any 8th-or-later value (e.g. the 8th owner) can't be selected at all.

TECHNICAL DETAIL

`values.slice(0,7)` caps the list; “Show more” is a plain `<span>` with no action. `src/main.js:8379, 8393`

FIX

Make it an expanding button (track expanded groups in state) or paginate.

P0-3 No way to see or clear active filters

UX · IN-PROGRESS WORK

WHAT IT MEANS

The old “active filters” bar (with per-filter remove chips and a “Clear all”) was removed. Users can end up with a narrowed list and no clear indication of why, and the only reset clears everything at once.

TECHNICAL DETAIL

`renderContactFilterBar` is still defined but never called; its `remove-contact-filter` handler is now dead code. `src/main.js:8410` (unused)

FIX

Restore a compact active-filter chip strip in the toolbar (reuse `activeContactFilters()`); delete the orphaned handlers.

P0-4 Contact import puts data in the wrong fields

CORRECTNESS · DATA CORRUPTION

WHAT IT MEANS

When importing a contacts spreadsheet, any blank cell shifts every following column one place to the left — so a phone number can land in the email field and be lost, silently.

TECHNICAL DETAIL

The row splitter `/(\?: "[^"]*" | [^, ])+/g` skips empty fields. Example: header `Name,Email,Phone` with row `John Doe, ,555-1234` → email becomes the phone number. `src/main.js:8320`

FIX

Route imports through the existing correct CSV parser `wbParseCsv` (`src/main.js:14365`). One change also fixes P2 items on multi-line fields and decimals.

P0-5 Malicious “app file” can run code for teammates

SECURITY · STORED XSS (HIGH)

WHAT IT MEANS

The Workspace App Builder lets a user install an app from a `.questapp.json` file. A crafted file can smuggle in a hidden script that then runs in every teammate's browser when they view the workspace — potentially stealing their session.

TECHNICAL DETAIL

Imported `color` values are copied verbatim (no format check) and interpolated into `style="..."` without escaping in ~6 places, allowing attribute break-out. `import src/main.js:14523; sinks 13536, 14031, 14278`

FIX

Validate every color against `^#[0-9a-fA-F]{3,8}$` at the import boundary; escape at the sinks. Enforcing CSP (P2-18) adds a second layer.

5 · Priority 1 — High severity 7

Real bugs and risks users or customers will encounter. Schedule for the next work cycle.

P1-6 The app rebuilds the entire screen on every change

PERFORMANCE · SYSTEMIC

WHAT IT MEANS

Every small update — a click, a background sync, a 15-second phone-status poll — throws away and rebuilds the whole page. This wastes performance, and it can interrupt users mid-typing and reset their scroll position. It is the single biggest lever in this report because it also causes several other problems.

TECHNICAL DETAIL

`render()` reassigns `app.innerHTML` for the entire shell + page + command palette, invoked from ~549 sites, plus a 15s poll, a 5s portal poll, and ~45 realtime table subscriptions. Also reboots the embedded Tasks iframe on background refresh. `src/main.js:2717`

FIX

Coalesce renders through a single animation-frame scheduler, and update only the changed region rather than the whole page. Highest ROI change in the codebase.

P1-7 “Today” is off by one day in the evening

CORRECTNESS · TIMEZONE

WHAT IT MEANS

For the US/Arizona team, after about 5pm the app starts treating tomorrow as “today.” The dashboard’s “today’s dispatch,” the mini-calendar highlight, and automation due-dates all drift by a day.

TECHNICAL DETAIL

`isoDate()` returns a UTC date while surrounding logic uses the local date. `src/main.js:38588`

FIX

Compute the local date in `isoDate()`, or standardize all “today” comparisons on one basis.

P1-8 Incoming text messages can vanish silently

CORRECTNESS · DATA LOSS (API)

WHAT IT MEANS

If two texts arrive from a brand-new number at nearly the same moment, one message can be dropped with no error and no retry — the customer’s reply simply disappears.

TECHNICAL DETAIL

On a contact-insert race the code falls back to a non-existent contact id, the message insert fails its foreign key unchecked, and the webhook still returns `{ok:true}` so the provider never retries.

`api/sms-inbound.js:46`

FIX

Re-select the existing contact on insert failure; check the message insert result and return a non-2xx so the provider retries.

P1-9 Saving portal notes can wipe existing ones

CORRECTNESS · DATA LOSS (API)

WHAT IT MEANS

When a client saves annotations in the portal, the system deletes all their existing notes first, then re-adds them. If the re-add fails midway, every note is gone with no recovery.

TECHNICAL DETAIL

Delete-then-reinsert with no transaction. `api/client-portal-annotations.js:71`

FIX

Perform the replace inside a single database transaction/RPC, or insert-then-delete-stale.

P1-10 Abuse protection is weaker than it appears

SECURITY · ANTI-AUTOMATION

WHAT IT MEANS

The rate limit that's supposed to stop attackers from guessing secret links or portal passwords resets often and can be sidestepped, so guessing attacks are more feasible than intended.

TECHNICAL DETAIL

The limiter is in-memory per serverless instance (effective limit = limit × warm instances, resets on cold start) and keys off the spoofable leftmost `X-Forwarded-For`. It is the sole control behind proposal/invite token enumeration and portal-password guessing. `api/_lib/rate-limit.js:3;`
`http-security.js:24`

FIX

Back the limiter with a shared store (e.g. Vercel KV / Redis / a Postgres table); use the platform-provided client IP.

P1-11 Keyboard users can't select or edit contacts

ACCESSIBILITY

WHAT IT MEANS

Someone using only a keyboard (including many screen-reader users) cannot tick a contact's checkbox, use inline edit, or reach the row menu. This is an accessibility and compliance gap.

TECHNICAL DETAIL

Row checkboxes are `tabindex="-1"` inside a plain `<span data-action>`; inline-edit fields are non-focusable spans; the row-menu dots and header chevron are dead affordances. `src/main.js:8112, 8515`

FIX

Use real `<button>` elements (or add role + tabindex + key handlers); make the checkbox itself focusable.

P1-12 Recurring tasks can reappear already overdue

CORRECTNESS · SCHEDULING

WHAT IT MEANS

If a repeating task is completed late, the next copy is created with a due date in the past. The user then has to clear weeks of "historical" tasks to catch up.

TECHNICAL DETAIL

The next occurrence anchors to the old due date and advances exactly one interval, with no roll-forward past today. `src/main.js:9009`

FIX

After computing the next date, advance whole intervals until it is on or after today.

6 · Priority 2 — Medium 11

Hardening, data hygiene, accessibility polish, and performance. Fold into related feature work.

Correctness & data

ID	ISSUE	DETAIL & LOCATION
P2-13	Multi-line spreadsheet cells corrupt import	CSV pre-split on newlines tears quoted multi-line fields. <code>main.js:8321</code>
P2-14	Exported CSV can run formulas in Excel	Cells starting <code>= + - @</code> not neutralized (CSV-injection). <code>main.js:14350</code>
P2-15	Price-book import 100× cost error	Strips all commas → comma-decimal locales mis-read. <code>main.js:5602</code>
P2-16	Portal writes report success on zero rows	PATCH/DELETE on a non-owned id returns “done”. Misreport, no data leak.
P2-17	Bad password hash returns 500 not 401	Unguarded <code>timingSafeEqual</code> . <code>api/client-portal-open.js:34</code>

Back-end hardening

ID	ISSUE	DETAIL & LOCATION
P2-18	Content-Security-Policy is report-only	Well-formed but never enforced — finish the staged rollout. <code>vercel.json:26</code>
P2-19	Portal session secret falls back to master key	If a dedicated secret is unset, sessions are signed with the service-role key. <code>portal-session.js:10</code>
P2-20	Stripe webhook leaks raw error text	Return generic messages, log details server-side.
P2-21	Upload bucket auto-created without type allowlist	Loses storage-layer MIME backstop on fresh environments.

Performance (all recomputed on every screen rebuild — compounds P1-6)

ID	ISSUE	DETAIL & LOCATION
P2-22	Same contact/task lists filtered 138× uncached	Memoize <code>companyX()</code> per render pass.
P2-23	Per-row workspace re-resolution (O(n-m))	Hoist invariants out of the per-element predicate. <code>main.js:34431</code>
P2-24	Name lookups scan the full list inside row loops	Build id→record Maps for <code>profileById</code> / <code>memberName</code> .

Accessibility & consistency

ID	ISSUE	DETAIL & LOCATION
P2-25	New rail hardcodes colors instead of theme tokens	~10 hex literals break theming/dark-mode. <code>styles.css:24247</code>
P2-26	Tiny grey counts fail contrast (WCAG AA)	<code>#a8a29e</code> at 8–10px $\approx$ 2.5:1. Darken/enlarge.
P2-27	Icon buttons labeled by tooltip only; missing button type	Add <code>aria-label</code> ; default 64 buttons to <code>type="button"</code> .
P2-28	Filter rail dominates the mobile screen	Below 1100px it stacks above the list; use a drawer/chips.

7 · Priority 3 — Architecture (OOAD) long-term

These are maintainability risks, not bugs. They make future changes slower and more error-prone. The recommended approach is incremental — no risky “big rewrite.”

In plain terms. Almost the entire application lives in one enormous file. It works, but it means every change carries a risk of accidentally breaking something unrelated, and it's hard to test in isolation. Tidying it up gradually will reduce bugs and let the team move faster.

Measured facts

ONE FILE, MANY JOBS

main.js = 38,781 lines holding routing, screens, data access, business rules, and events together (~2,073 top-level symbols).

SHARED MUTABLE STATE

228 fields in one global object, referenced **2,435×** — any function can change anything.

GIANT DISPATCHERS

handleAction = 2,055 lines / ~309 branches; a second is 529 lines. Adding a feature means appending to these.

DATABASE CALLS IN THE UI

252 table calls + **42** stored-procedure calls inline in the screen code — no data-access layer.

TESTS

52 of 96 test files check that certain *text* exists in the code, not that it works — they pass even if the feature is broken.

PLUGIN SYSTEM

Adding a plugin requires editing the monolith *and* a hand-copied database rule — the source of a known drift bug.

Recommended single highest-leverage move. Introduce a **per-feature “registry”** so each feature owns its own actions, screen, and data in a small module instead of adding to the giant shared functions. This can be done gradually, one feature at a time, and it simultaneously (a) creates safe boundaries, (b) makes the plugin system properly extensible, and (c) finally makes real (behavioral) testing possible. It is fully compatible with the team's existing decision to avoid a big-bang split.

8 · What's already done well

An audit should be balanced. These are verified strengths worth protecting as the code evolves.

- **Tenant isolation is enforced twice** — database row-level security for logged-in users, plus explicit company scoping on server paths. No way found for one company to read another's data.
- **Database functions are uniformly hardened** — all 111 privileged functions across 37 migrations set a fixed search path; sensitive ones revoke public access and re-check permissions server-side.
- **Payment & messaging webhooks are properly verified** — Stripe signatures and cron secrets use correct, constant-time checks.
- **A recorded past privilege-escalation hole was correctly closed**, and invites fail safe (any elevated role request is downgraded to "member").
- **Output is consistently escaped** in the browser via a correct helper, and heavy libraries (maps, PDF, zip) are loaded only when needed.
- **No secrets are committed** to the repository.
- **The code documents its own reasoning inline** and maintains a vendor-neutral "project brain," which made this audit faster and safer.

9 • Recommended sequence

STAGE	ITEMS	OUTCOME
Now	P0-1 ... P0-5	Finishes the current contact-rail work correctly and closes the data-corruption and stored-XSS paths. Small, self-contained.
Next cycle	P1-6 ... P1-12	Fix the date bug and the two data-loss paths (small); plan the screen-rebuild improvement (P1-6) as its own focused effort.
Ongoing	P2-13 ... P2-28	Hardening and accessibility, folded into whatever feature touches those areas.
Continuous	Architecture	Extract one feature at a time into the registry pattern, each with real tests.

Immediate next step available. The five P0 items can be delivered together as one reviewable change: wire or remove the dead rail controls, restore the active-filter summary, route imports through the correct CSV parser, and add the color-import safety check — then re-run the build and tests.

Every finding in this report was confirmed against the current source code. Locations are given as `file:line` for engineers. One issue (a broken stylesheet rule) was fixed during the audit and verified with a clean production build. No production data, secrets, or customer records are included in this document.